

СТАНДАРТНЫЕ СРЕДСТВА

Семинар 6

pair, map

std::pair

- заголовочный файл <utility> – <https://en.cppreference.com/w/cpp/utility/pair>
- std::pair – шаблон класса, который позволяет хранить два разнородных объекта как единое целое
- `template<class T1, class T2> struct pair;`

std::pair

Поля:

- first – получение первого элемента пары,
- second – получение второго элемента пары.

Non-member functions:

- make_pair() – создать пару.

std::pair

```
1  #include <iostream>
2  #include <utility>
3  #include <string>
4  #include <vector>
5
6  int main(int argc, char** argv)
7  {
8      std::pair<std::string, std::vector<unsigned>> pMisp;
9      std::vector<unsigned> vDone{3, 4, 0, 1}; // количество зачтенных задач
10     pMisp = std::make_pair("Ivanov", vDone);
11     std::cout << pMisp.first << " ";
12     for (int i = 0; i < pMisp.second.size(); ++i)
13     |     std::cout << pMisp.second[i] << " ";
14     std::cout << std::endl;
15     return 0;
16 }
```

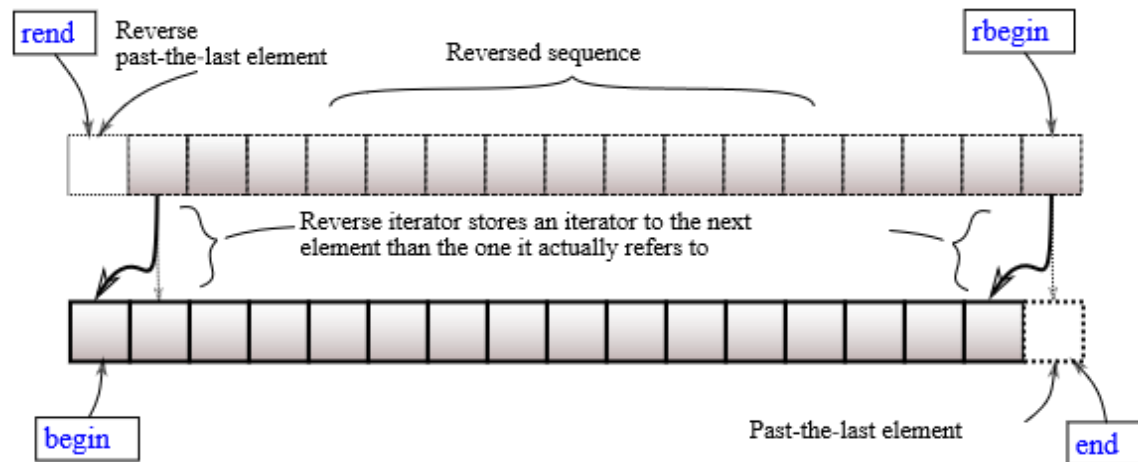
```
shulginaali@075-AShulgina:~/cpp$ ./a.out
Ivanov 3 4 0 1
```

std::map

- заголовочный файл <map> – <https://en.cppreference.com/w/cpp/container/map>
- ассоциативный контейнер, который работает по принципу – [ключ – значение],
- ключом может быть все что угодно, от переменной до класса,
- при добавлении нового элемента, контейнер будет отсортирован по возрастанию.

std::map – итераторы

```
std::map<int, std::string> _map;  
std::map<int, std::string>::iterator it = _map.begin();  
for (it; it != _map.end(); ++it)  
{  
    /// do smth  
}
```



std::map – итераторы

`std::map<T1, T2>::iterator it;` – объявление итератора

`it->first` – получение ключа

`it->second` – получение значения

К итератору нельзя применять никакие арифметические операции, для увеличения или уменьшения используется инкремент или декремент:

`it++;`

`it--;`

Ключевое слово auto

- для переменных указывает, что тип объявляемой переменной будет автоматически выведен из ее инициализатора
- = выводом типа или «автоматическим определением типа данных компилятором»

```
// 4.0 - это литерал типа double, поэтому и переменная x должна быть типа double
auto x = 4.0;
// выражение 3 + 4 обрабатывается как целочисленное, поэтому и переменная y должна быть типа int
auto y = 3 + 4;
```

```
1 // C++03
2 for (std::vector<std::map<int, std::string>>::const_iterator it = container.begin(); it != container.end(); ++it)
3 {
4     // do smth
5 }
6 // C++11
7 for (auto it = container.begin(); it != container.end(); ++it)
8 {
9     // do smth
10 }
```


std::map

Доступ к элементу:

- at – доступ к элементу
- operator[] – доступ или добавление элемента

```
std::map<std::string, int> _map = {"one", 1}, {"six", 6}, {"five", 5};  
// at() выработает исключение std::out_of_range, т.к. такого ключа нет  
//std::cout << _map.at("nine") << std::endl;  
std::cout << _map["nine"] << std::endl;  
std::cout << _map.at("nine") << std::endl;  
std::cout << _map["six"] << " " << _map.at("six") << std::endl;
```

```
chernousov@DESKTOP-Igor:~/ex$ ./a.out  
0  
0  
6 6
```

```
chernousov@DESKTOP-Igor:~/ex$ ./a.out  
terminate called after throwing an instance of 'std::out_of_range'  
what(): map::at  
Aborted
```

std::map

Контейнер:

- `empty` – проверить контейнер на пустоту
- `size` – получить количество элементов контейнера

```
std::map<std::string, int> _map = {  
    {"one", 1},  
    {"six", 6},  
    {"five", 5}  
};  
std::cout << _map.size() << " " << _map.empty() << std::endl;
```

```
chernousov@DESKTOP-Igor:~/ex$ ./a.out  
3 0
```

std::map

Модификаторы:

- clear — очистить контейнер
- insert — вставить новый элемент
- erase — удалить элемент

```
std::map<std::string, int> _map = {  
    {"one", 1},  
    {"six", 6},  
    {"five", 5}  
};  
  
_map.clear();  
std::cout << "Size: " << _map.size() << " empty: " << _map.empty() << std::endl;  
_map.insert({"one", 1}, {"six", 6});  
_map.insert(_map.end(), {"nine", 9});  
  
std::cout << "Size: " << _map.size() << " empty: " << _map.empty() << std::endl;  
  
_map.erase("six");  
std::cout << "Size: " << _map.size() << " empty: " << _map.empty() << std::endl;
```

```
chernousov@DESKTOP-Igor:~/ex$ ./a.out  
Size: 0 empty: 1  
Size: 3 empty: 0  
Size: 2 empty: 0
```

std::map

Поиск:

- count – получить количество элементов с заданным ключом, для map вернет 0 или 1
- find – узнать, если ли элемент в контейнере по ключу и получить итератор на него

```
std::map<std::string, int> _map = {  
    {"one", 1},  
    {"six", 6},  
    {"five", 5}  
};  
std::cout << "Count: " << _map.count("one") << " " << _map.count("nine") << std::endl;  
std::map<std::string, int>::iterator it = _map.find("one");  
if (it != _map.end())  
    std::cout << "key one found " << it->second << std::endl;  
  
it = _map.find("nine");  
if (it != _map.end())  
    std::cout << it->second << std::endl;  
else  
    std::cout << "key nine not found" << std::endl;
```

```
chernousov@DESKTOP-Igor:~/ex$ ./a.out  
Count: 1 0  
key one found 1  
key nine not found
```

Range-based циклы

Выполняет цикл `for` по диапазону.

Используется как более понятный эквивалент традиционного цикла `for`, работающего с диапазоном значений, например, со всеми элементами в контейнере.

```
int math[] = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41};  
for (int number : math) // итерация по массиву math  
    std::cout << number << ' '; // получение доступа к элементу массива в этой итерации через переменную number
```

```
int math[] = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41};  
for (auto number : math)  
    std::cout << number << ' ';
```

В таком случае, каждый обработанный элемент копируется в переменную `number`. Избежать этого можно сославшись на исходный элемент с помощью ссылки:

```
int math[] = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41};  
for (auto& number : math)  
    std::cout << number << ' ';
```

std::map

```
shulginaali@075-Ashulgina:~/cpp$ ./a.out
1 one      2 two      3 three    4 four      5 five      6 six      8 eight      9 nine
1 one      2 two      3 three    5 five      6 six      8 eight      9 nine
1 one      2 two      3 three    5 five      6 six      7 seven     8 eight      9 nine
three
```

```
1 #include <iostream>
2 #include <string>
3 #include <map>
4 #include <iomanip>
5
6 void print(const std::map<int, std::string> &m)
7 {
8     // Range-based for-loop
9     for (const auto& it : m)
10     {
11         std::cout << std::setw(4) << it.first << " " << it.second << "\t";
12     }
13     std::cout << "\n";
14 }
```

```
6 void print(const std::map<int, std::string> &m)
7 {
8     for (auto it = m.begin(); it != m.end(); ++it)
9     {
10         std::cout << std::setw(4) << it->first << " " << it->second << "\t";
11     }
12     std::cout << "\n";
13 }
```

```
15 int main(int argc, char **argv)
16 {
17     std::map<int, std::string> _map =
18     {
19         {1, "one" }, {2, "two" }, {3, "three"},
20         {4, "four"}, {5, "five"}, {6, "six" }
21     };
22     _map.insert({{8, "eight"}, {9, "nine"}});
23     print(_map);
24     std::map<int, std::string>::iterator it = _map.find(4);
25     if (it != _map.end())
26     {
27         _map.erase(it);
28         print(_map);
29     }
30     it = _map.find(6);
31     if (it != _map.end())
32     {
33         ++it;
34         _map.insert(it, std::make_pair(7, "seven"));
35         print(_map);
36     }
37     std::cout << _map[3] << std::endl;
38     return 0;
39 }
```

Задачи

В файле построчно записаны значения вида:

<Фамилия_студента номер_студенческого средний_балл>.

Считать данные в `std::map`, при чтении повторяющегося ключа значение не должно быть перезаписано.

Удалить студентов из `std::map` у которых средний балл менее 3.0.

Вывести полученные значения на экран.

```
std::map<std::string, std::pair<int, double>> _map;
```